

Introduction to Deep Learning

Hoa T. Vu

The Neuron

A Single Neuron

Weighted sum + **activation function**:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \mathbf{w}^\top \mathbf{x} + b$$

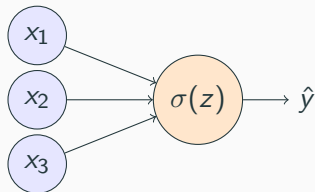
$$\hat{y} = \sigma(z)$$

A Single Neuron

Weighted sum + **activation function**:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = \mathbf{w}^\top \mathbf{x} + b$$

$$\hat{y} = \sigma(z)$$



$\mathbf{w}$ weights: learned parameters

b bias: shifts the threshold

σ activation: adds nonlinearity

Why Activation Functions?

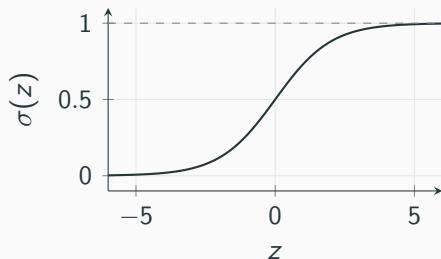
Without activations, stacking layers collapses to one linear map:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = W'\mathbf{x} + \mathbf{b}'$$

Activations add nonlinearity — letting the network represent complex functions.

Sigmoid

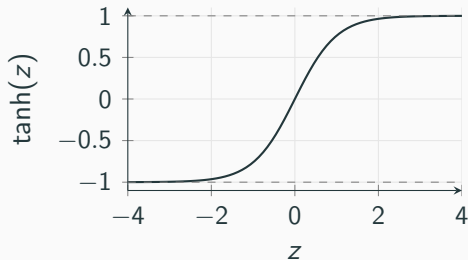
$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- Output $\in (0, 1)$ — probability
- **Saturates:** gradient ≈ 0 for large $|z|$
- Output layer for binary classification

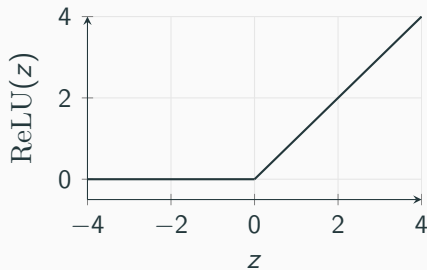
Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



- Output $\in (-1, 1)$
- **Zero-centered**
- Still saturates; less common today

$$\text{ReLU}(z) = \max(0, z)$$



- Gradient: 1 for $z > 0$, 0 otherwise
- No saturation on positive side
- **Default** for hidden layers

Which Activation to Use?

Activation	Range	Typical use
Sigmoid	$(0, 1)$	Output: binary classification
Softmax	$(0, 1)^k$, sums to 1	Output: multi-class
Tanh	$(-1, 1)$	RNNs, some older networks
ReLU	$[0, \infty)$	Hidden layers (default)

Rule of thumb

Use ReLU for hidden layers. Match the output activation to the task.

Matrix Notation

A Layer as Matrix Multiplication

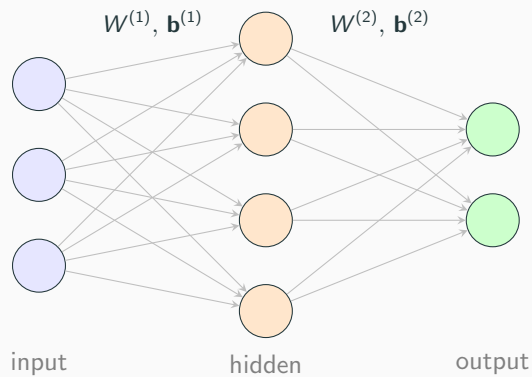
$$\mathbf{z} = W\mathbf{x} + \mathbf{b}, \quad \mathbf{a} = \sigma(\mathbf{z})$$

Each row of W : weights of one neuron.

$$\underbrace{W}_{m \times n} \underbrace{\mathbf{x}}_{n \times 1} + \underbrace{\mathbf{b}}_{m \times 1} = \underbrace{\mathbf{z}}_{m \times 1}$$

$$W = \begin{pmatrix} w_{11} & \cdots & w_{1n} \\ \vdots & \ddots & \vdots \\ w_{m1} & \cdots & w_{mn} \end{pmatrix}$$

A Two-Layer Network



A Deep Network

Stack L layers. Layer ℓ computes:

$$\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \quad \mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$$

with $\mathbf{a}^{(0)} = \mathbf{x}$ (the input).

Mini-batch Processing

Stack B inputs as columns of $X \in \mathbb{R}^{n \times B}$:

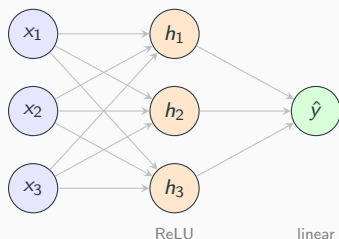
$$Z = WX + \mathbf{b}\mathbf{1}^\top \in \mathbb{R}^{m \times B}$$

All B samples processed with one matrix multiply.

Why GPUs matter

GPUs excel at large matrix multiplications — mini-batch training is orders of magnitude faster than one sample at a time.

Exercise: Forward Pass



Input: $\mathbf{x} = (1, 0, -1)^\top$

Layer 1 $W^{(1)}, \mathbf{b}^{(1)}$:

$$W^{(1)} = \begin{pmatrix} 1 & 2 & 0 \\ -1 & 1 & 1 \\ 0 & -1 & 2 \end{pmatrix}, \quad \mathbf{b}^{(1)} = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$$

Layer 2 $W^{(2)}, b^{(2)}$:

$$W^{(2)} = \begin{pmatrix} 1 & -2 & 1 \end{pmatrix}, \quad b^{(2)} = 0$$

Loss function

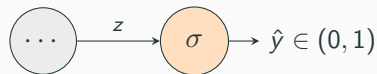
Loss Function: Binary Classification

$\hat{y} = \sigma(z) \in (0, 1)$, label $y \in \{0, 1\}$.

Binary cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

- $y = 1, \hat{y} \approx 0$: loss $\rightarrow \infty$
- $y = 1, \hat{y} \approx 1$: loss ≈ 0



1 output node
sigmoid

Loss Function: Multi-class Classification

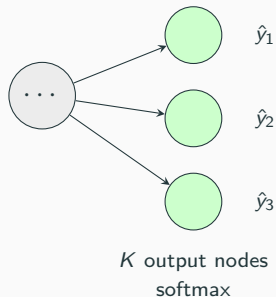
Softmax turns logits into probabilities:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_k \hat{y}_k = 1$$

Cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log \hat{y}_{y_i}$$

Only the correct class's probability is penalized.



Exercise: Cross-Entropy Loss (1/3)

A 3-class classifier produces logits $\mathbf{z} = (2, 1, 0)^\top$. The true label is class 1.

Step 1. Apply softmax to get predicted probabilities $\hat{\mathbf{y}}$:

$$\hat{y}_k = \frac{e^{z_k}}{e^{z_1} + e^{z_2} + e^{z_3}} = \frac{e^{z_k}}{e^2 + e^1 + e^0}$$

Compute $\hat{y}_1, \hat{y}_2, \hat{y}_3$. Verify that $\hat{y}_1 + \hat{y}_2 + \hat{y}_3 = 1$.

Exercise: Cross-Entropy Loss (2/3)

Using your softmax probabilities from the previous step:

Step 2. Compute the cross-entropy loss.

The true label is class 1, so only $\hat{y}_1$ appears in the loss:

$$\mathcal{L} = -\log \hat{y}_1$$

Is the loss large or small? What does that tell you about how confident the model is in the correct class?

Exercise: Cross-Entropy Loss (3/3)

Step 3. Reason about confidence.

Compute $\mathcal{L} = -\log \hat{y}_1$ for each set of logits and compare:

Logits $\mathbf{z}$	$\hat{y}_1$	$\mathcal{L}$
$(2, 1, 0)^\top$	?	?
$(10, 0, 0)^\top$	?	?
$(1, 1, 1)^\top$	?	?

What pattern do you observe as the model becomes more or less confident in the correct class?

Loss Function: Regression

Output: $\hat{y} \in \mathbb{R}$, label $y \in \mathbb{R}$.

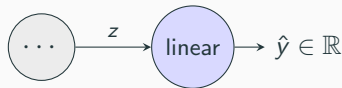
Mean squared error (MSE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2$$

Mean absolute error (MAE):

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

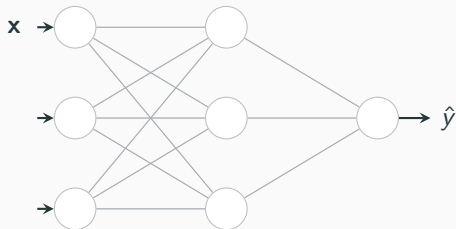
MSE penalizes large errors more; MAE is more robust to outliers.



1 output node
no activation

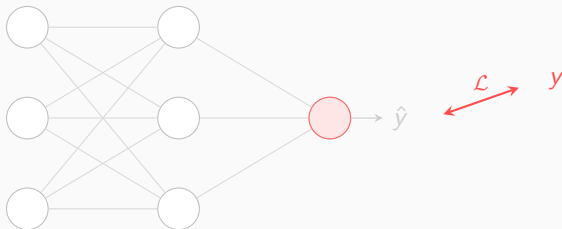
Training a Neural Network

Step 1: Forward Pass



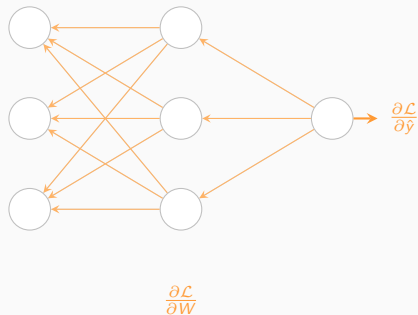
Run the input through the network layer by layer to produce a prediction.

Step 2: Compute Loss



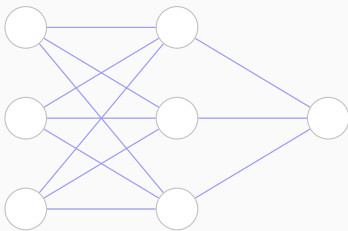
Compare $\hat{y}$ to the true label y . The loss $\mathcal{L}$ measures the error.

Step 3: Backward Pass



Propagate gradients *backwards*. Each weight learns how much it contributed to the error.

Step 4: Update Weights



$$W \leftarrow W - \alpha \nabla_W \mathcal{L}$$

Nudge every weight a small step in the direction that reduces the loss.

The Training Loop: Setup

Define the model, loss, and optimizer once before training.

```
import torch
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(3, 4),
    nn.ReLU(),
    nn.Linear(4, 1),
)

loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

The Training Loop: One Epoch

One **epoch** = one full pass over the training data.

```
for X_batch, y_batch in dataloader:    # mini-batches
    # 1. forward pass
    y_hat = model(X_batch)

    # 2. compute loss
    loss = loss_fn(y_hat, y_batch)

    # 3. backward pass
    optimizer.zero_grad()    # clear old gradients
    loss.backward()          # compute new gradients

    # 4. update weights
    optimizer.step()
```

The Training Loop: Full Loop

Repeat for many epochs; monitor validation loss to stop early.

```
for epoch in range(num_epochs):
    model.train()
    for X_batch, y_batch in dataloader:
        y_hat = model(X_batch)
        loss = loss_fn(y_hat, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    # evaluate
    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(X_val), y_val)
    print(f"epoch {epoch}: val loss = {val_loss:.4f}")
```

Dropout

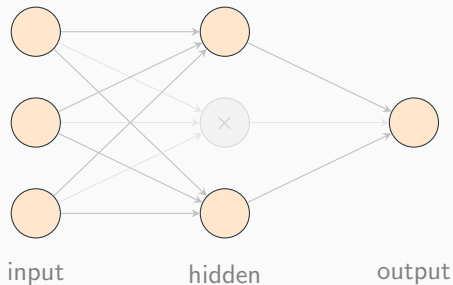
The Problem: Overfitting

Large networks easily memorize training data and fail on new inputs.

Dropout is a simple technique to fight this.

Dropout: The Idea

Each training step: **randomly zero out** neurons with probability p .



A different subset is dropped each step.

Why Dropout Works

- **Forces redundancy** — no neuron can be relied on exclusively.
- **Ensemble effect** — each step trains a slightly different network.
- **Test time:** dropout off; outputs scaled by $(1 - p)$.

Common values

$p = 0.5$ for hidden layers; $p = 0.1$ – 0.2 near the input.

Dropout in PyTorch

```
model = nn.Sequential(  
    nn.Linear(3, 64),  
    nn.ReLU(),  
    nn.Dropout(p=0.5),    # dropped during training only  
    nn.Linear(64, 1),  
)
```

```
model.train()    # dropout ON  
model.eval()     # dropout OFF
```

Convolutional Neural Networks

Why Not Just Use a Feedforward Net?

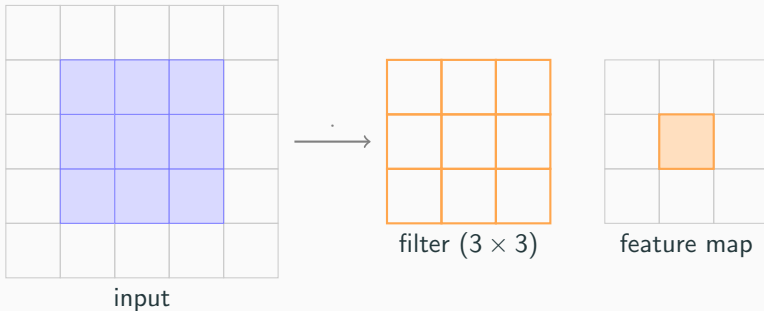
28×28 image $\rightarrow$ 784 inputs. One FC layer with 256 neurons = 200,704 weights.

- Same patterns (edges, corners) appear at many positions — redundant weights.
- No built-in **spatial structure**.

Fix: share a small **filter** across all positions.

The Convolution Operation

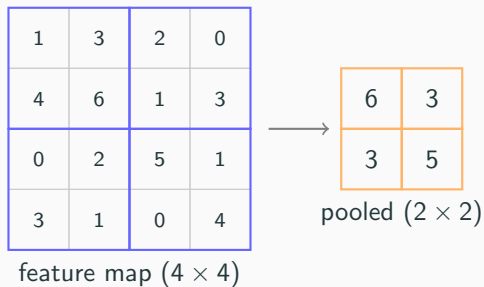
Slide a 3×3 filter across the input; dot product at each position $\rightarrow$ feature map.



Same filter weights at every position — far fewer parameters.

Pooling

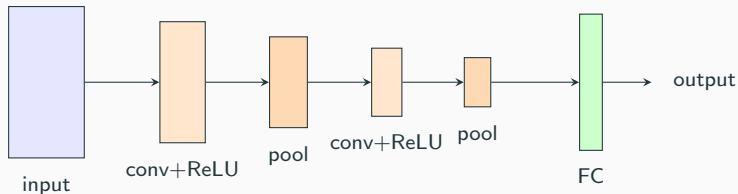
Max pooling: take the max in each 2×2 block.



Reduces spatial size; retains the strongest activations.

Putting It Together

Conv + ReLU + pool, repeated; then flatten for a classifier.



- Early layers detect edges and textures
- Later layers combine them into shapes and objects

CNN in PyTorch

```
model = nn.Sequential(  
    nn.Conv2d(1, 16, kernel_size=3, padding=1),  # 1 channel in, 16  
        out  
    nn.ReLU(),  
    nn.MaxPool2d(2),          # halve spatial dims  
  
    nn.Conv2d(16, 32, kernel_size=3, padding=1),  
    nn.ReLU(),  
    nn.MaxPool2d(2),  
  
    nn.Flatten(),  
    nn.Linear(32 * 7 * 7, 10),  # 10 classes  
)
```

Input: $1 \times 28 \times 28$ (grayscale image, e.g. MNIST).